# UAVLogBook

## Flight Analysis Platform

Complete Installation & Setup Guide

Version 1.0 — June 2025

**Server Install**
PHP + MySQL
cPanel Hosting

**Web App Build**
React + Vite
Static Hosting

**Mobile App Build**
React Native
iOS + Android

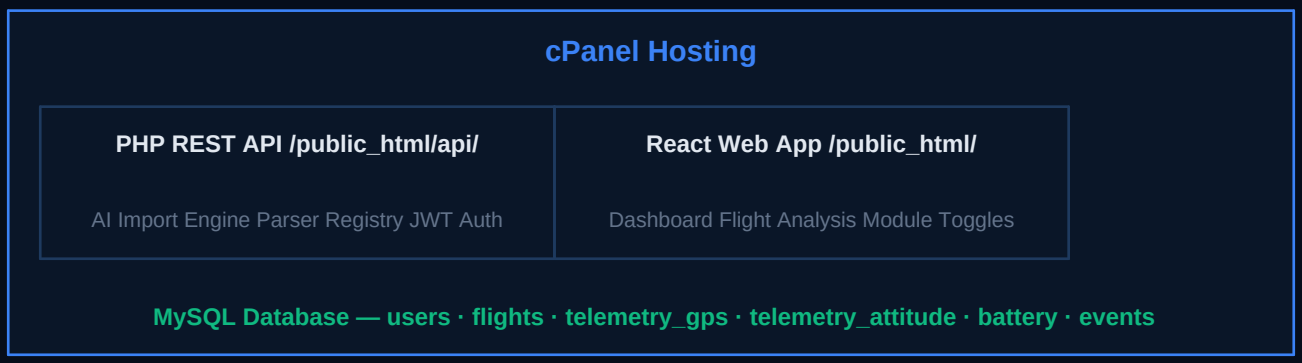
**AI Import Engine**
Claude AI
Auto-detect

# Table of Contents

# Section 1 — Overview & Architecture

## System Overview

UAVLogBook is a full-stack platform for importing, storing, and analysing UAV flight logs from any autopilot or drone system. The architecture is designed to run entirely on shared cPanel hosting — no VPS or Docker required.




Mobile clients (iOS and Android) built with React Native connect to the same PHP API over HTTPS. The AI import engine sends file samples to the Claude API for format detection — all heavy parsing happens server-side.

# Section 2 — Prerequisites

## Server Requirements

| Component | Minimum | Recommended | Notes |
|---|---|---|---|
| PHP | 7.4 | 8.2+ | With PDO, curl, mbstring, json |
| MySQL / MariaDB | 5.7 / 10.3 | 8.0 / 10.6+ | InnoDB engine required |
| Web Server | Apache 2.4 | Apache 2.4+ | mod_rewrite must be enabled |
| Disk Space | 500 MB | 5 GB+ | For storing log files |
| Memory (PHP) | 128 MB | 256 MB+ | memory_limit in php.ini |
| Upload Size | 500 MB | 500 MB | upload_max_filesize in php.ini |
| SSL Certificate | Required | Let's Encrypt | HTTPS mandatory for mobile |
| SSH Access | Recommended | Recommended | For installer script |

## Developer Workstation (for building)

| Tool | Version | Purpose | Download |
|---|---|---|---|
| Node.js | 18+ | Build React web app | nodejs.org |
| npm | 9+ | Package manager | Included with Node.js |
| Git | Any | Clone / manage code | git-scm.com |
| Expo CLI | Latest | Build mobile app | npm install -g expo-cli |
| EAS CLI | Latest | Build iOS/Android binaries | npm install -g eas-cli |
| FTP Client | Any | Upload files to cPanel | FileZilla (free) |
| Expo Account | Free | Required for EAS builds | expo.dev |
| Apple Dev Acc. | - | iOS App Store (optional) | developer.apple.com |
| Google Play | - | Android Play Store (opt.) | play.google.com/console |

# Section 3 — Server Installation (cPanel)

## Method A — Automated Installer (Recommended)

Connect to your cPanel hosting via SSH, upload the UAVLogBook package, then run the installer script. The script handles directory creation, config writing, and database import automatically.

### 1 Connect to cPanel via SSH

- Log in to cPanel → Terminal, or use an SSH client:
- $ ssh username@yourdomain.com

### 2 Upload the package

- Option A — via cPanel File Manager: upload UAVLogBook.zip, then extract.
- Option B — via command line:

```
$ cd ~/
$ unzip UAVLogBook.zip
$ cd uavlogbook
```

### 3 Run the installer

- Make the script executable and run it:

```
$ chmod +x installer/install_server.sh
$ bash installer/install_server.sh
```

### 4 Follow the prompts

- The installer will ask for: MySQL host, database name, username,
- password, your domain URL, and (optionally) Anthropic API key.
- All values are saved to a secure config file outside the web root.

## Method B — Manual Installation

Use this method if you have no SSH access (shared hosting only).

### 1 Create database in cPanel

- Go to cPanel → MySQL Databases
- Create a new database (e.g. youraccount_uavlogbook)
- Create a database user with a strong password

- Grant ALL PRIVILEGES to the user on the database
- Note the host (usually 'localhost'), database name, username, password

## 2 Import database schema

- Go to cPanel → phpMyAdmin
- Select your new database in the left sidebar
- Click the Import tab
- Choose file: server/db/schema.sql
- Click Go — all tables will be created

## 3 Upload server files via FTP/File Manager

- Connect with FileZilla or use cPanel File Manager:
- Upload the entire server/ directory to: public_html/api/
- Upload .htaccess to: public_html/.htaccess

## 4 Create config file

- In cPanel File Manager, navigate to public_html/api/config/
- Create a new file: config.local.php
- Paste the template below (replace ALL_CAPS values):

```
<?php
define('DB_HOST',   'localhost');
define('DB_NAME',   'youracct_uavlogbook');
define('DB_USER',   'youracct_dbuser');
define('DB_PASS',   'YOUR_STRONG_PASSWORD');
define('JWT_SECRET','YOUR_64_CHAR_RANDOM_STRING');
define('UPLOAD_DIR', '/home/youracct/uploads/logs/');
define('ANTHROPIC_API_KEY', 'sk-ant-...');
define('ALLOWED_ORIGINS', ['https://yourdomain.com']);
define('MAX_FILE_MB', 500);
define('TELEM_MAX_POINTS', 5000);
define('AI_MODEL', 'claude-sonnet-4-20250514');
define('APP_VERSION', '1.0.0');
define('API_VERSION', 'v1');
```

## 5 Create uploads directory

- Via cPanel File Manager, create directory:
- /home/youraccount/uploads/logs/
- Set permissions to 755 (Permissions → Change Permissions)

**IMPORTANT — PHP Upload Limits**

cPanel's default upload_max_filesize is often 2MB — far too small for flight logs. In cPanel → Select PHP Version → PHP Options, set: upload_max_filesize = 512M, post_max_size = 512M, memory_limit = 256M, max_execution_time = 300

# Section 4 — Database Schema Reference

## Core Tables

| | |
|---|---|
| `users` | User accounts, JWT tokens, display settings, view preferences |
| `aircraft` | UAV registry — make, model, firmware, serial number per user |
| `flights` | Master flight record — metadata, parse status, AI analysis, stats |
| `telemetry_gps` | GPS track: lat, lng, altitude, speed, satellites, HDOP, fix type |
| `telemetry_attitude` | Roll, pitch, yaw (degrees) and angular rates over time |
| `telemetry_battery` | Voltage, current, remaining %, consumed mAh, temperature |
| `telemetry_imu` | Accelerometer X/Y/Z, gyroscope X/Y/Z, vibration levels |
| `telemetry_rc` | RC input channels 1–12 and RSSI over time |
| `flight_events` | Arming, mode changes, failsafes, warnings, errors timeline |
| `annotations` | Pilot-added notes pinned to specific time or GPS position |
| `user_view_prefs` | Per-user module on/off state and dashboard layout positions |
| `share_tokens` | Secure tokens for sharing individual flights publicly |

**Storage Efficiency**

Telemetry tables are partitioned by flight_id (8 partitions) for fast queries. The TELEM_MAX_POINTS setting (default 5000) controls downsampling — a 10-minute ArduPilot BIN at 400Hz is downsampled to 5000 points per channel. Increase for higher resolution at the cost of more storage.

### Estimated Storage per Flight

| Log Length | Raw File | GPS Points | Attitude Points | DB Total (est.) |
|---|---|---|---|---|
| 5 minutes | 2–10 MB | 5,000 | 5,000 | ~2 MB |
| 30 minutes | 10–80 MB | 5,000 | 5,000 | ~8 MB |
| 2 hours | 50–400 MB | 5,000 | 5,000 | ~20 MB |

# Section 5 — Web App Build & Deploy

## Building the React Web App

The web app is a React + Vite single-page application. Build it once on your workstation, then upload the compiled static files to cPanel. No Node.js is needed on the server.

### 1 Install Node.js (if not already installed)

- Download from nodejs.org — choose LTS version (18+)
- Windows: run the .msi installer | macOS: use the .pkg installer
- Linux: sudo apt install nodejs npm (Ubuntu/Debian)

```
$ node --version   # should show v18.x.x or higher
$ npm --version    # should show 9.x.x or higher
```

### 2 Navigate to the web directory

```
$ cd uavlogbook/web
```

### 3 Configure your API URL

- Copy the example environment file and edit it:

```
$ cp .env.example .env.local
# Open .env.local in a text editor and set:
VITE_API_URL=https://yourdomain.com/api/v1
```

### 4 Install dependencies

- This downloads all required packages (~200MB, one-time):

```
$ npm install
```

### 5 Build the production bundle

- Creates an optimised bundle in the web/dist/ folder:

```
$ npm run build
# Output: web/dist/  (index.html + assets/)
```

### 6 Upload to cPanel

- Via cPanel File Manager:
- • Navigate to public_html/
- • Delete any existing index.html
- • Upload ALL files from web/dist/ directly into public_html/
- • The .htaccess file should already be there from server install
- Via FTP (FileZilla):

```
Local:  web/dist/*
Remote: /public_html/   (upload all files here, not in a subfolder)
```

**Windows users**

Use the included installer\build_web.ps1 script. Right-click it in File Explorer and choose 'Run with PowerShell'. It will prompt you to set the API URL and then build automatically. The dist\ folder will be ready to upload.

## Verifying the Web App

After uploading, open https://yourdomain.com in your browser. You should see the UAVLogBook login screen. Register a new account and log in.

If you see a blank page, check: (1) all files from dist/ were uploaded to public_html/ root, (2) the .htaccess file is present, (3) mod_rewrite is enabled in cPanel.

# Section 6 — Mobile App Setup (iOS & Android)

## About the Mobile App

The mobile app is built with React Native and Expo. It connects to the same PHP API as the web app. You can run it in development mode immediately, or build release binaries (.ipa for iOS, .apk/.aab for Android) using Expo Application Services (EAS).

### Step 1 — Configure API URL

Before building, set your server URL in the mobile app source:

```
# Open: uavlogbook/mobile/src/api.js
# Change line 6:
export const API_BASE_URL = 'https://yourdomain.com/api/v1';
# Replace yourdomain.com with your actual domain
```

### Step 2 — Install Dependencies

```
$ cd uavlogbook/mobile
$ npm install
$ npm install -g expo-cli eas-cli   # install CLI tools globally
```

### Step 3A — Development Testing (quick)

Test on your phone immediately using the Expo Go app (free, no Apple/Google account needed):

```
$ npx expo start
# A QR code will appear in the terminal
# Install 'Expo Go' from App Store or Google Play
# Scan the QR code with your phone's camera
# The app will load on your device
```

> **Expo Go Limitation**
> Expo Go works for testing but cannot access all native APIs. For production use, build a standalone binary using EAS Build (Step 3B).

### Step 3B — Production Build with EAS (recommended)

```
# 1. Create a free account at expo.dev
$ eas login

# 2. Configure the project (first time only)
$ eas build:configure

# 3. Build Android APK (for sideloading / testing)
$ eas build --platform android --profile preview

# 4. Build Android AAB (for Google Play Store)
$ eas build --platform android --profile production

# 5. Build iOS IPA (requires Apple Developer account $99/yr)
$ eas build --platform ios --profile production

# Builds run in the cloud (~10-20 minutes)
# Download links emailed when complete
```

## Step 4 — Install on Device

| Platform | Method | Steps |
|---|---|---|
| Android | Direct APK install | Download APK → enable 'Install unknown apps' in Settings → open APK → Install |
| Android | Google Play Store | Upload AAB to Google Play Console → create internal test → distribute |
| iOS | TestFlight | Upload IPA via Transporter app → add testers in App Store Connect |
| iOS | App Store | Submit for review via App Store Connect (requires Apple Developer account) |

# Section 7 — Configuration Reference

## config.local.php — All Settings

| | | |
|---|---|---|
| **DB_HOST** | `localhost` | MySQL server hostname. Usually 'localhost' on cPanel. |
| **DB_NAME** | `youracct_uavlog` | MySQL database name as created in cPanel. |
| **DB_USER** | `youracct_dbuser` | MySQL username with full privileges on DB_NAME. |
| **DB_PASS** | `StrongPassword!` | MySQL password. Use a strong random password. |
| **JWT_SECRET** | `64-char random hex` | Secret key for signing JWT tokens. NEVER share this. |
| **JWT_EXPIRY_HOURS** | `72` | How long login tokens stay valid. Default 72 hours. |
| **UPLOAD_DIR** | `/home/user/uploads/` | Absolute path for storing raw log files. Must be writable. |
| **MAX_FILE_MB** | `500` | Maximum upload size in megabytes per file. |
| **ANTHROPIC_API_KEY** | `sk-ant-...` | Claude API key for AI format detection. Leave blank to disable AI. |
| **AI_MODEL** | `claude-sonnet-4-...` | Claude model ID. Do not change unless instructed. |
| **TELEM_MAX_POINTS** | `5000` | Max telemetry data points stored per channel per flight. |
| **ALLOWED_ORIGINS** | `['https://your.com']` | Array of allowed CORS origins. Include your domain + localhost. |

**Security Note**

config.local.php is set to permissions 600 (readable only by owner) by the installer. Never put it inside public_html root — the installer places it in api/config/ which is protected by .htaccess rules that block direct PHP file access.

## Section 8 — Supported Log Formats

**ArduPilot DataFlash** `.BIN` ArduPilot / APM

Self-describing binary. FMT messages define all message types. Full telemetry: GPS, ATT, BATT, IMU, VIBE, RCIN, MODE, EV, MSG, PARM.

**MAVLink Telemetry** `.TLOG` Mission Planner / QGC

Binary stream of MAVLink v1/v2 packets. Recorded by ground station software. Contains GPS, attitude, battery, RC input, status text messages.

**PX4 ULog** `.ULG / .ULOG` PX4 / Pixhawk

Binary format with magic header 0x55 0x4C 0x6F 0x67. Self-describing topics: vehicle_gps_position, vehicle_attitude, battery_status.

**Skyline** `.SKYLOG` Skyline GCS

Brace-delimited text log with embedded binary tlm packets. GPS, altitude, speed, battery voltage, SBUS RC channels, statustext events, home position. Full reverse-engineered format — see Section 8a for complete specification.

**DJI TXT Record** `.TXT` DJI GO / Fly / Pilot

Binary format despite .txt extension (reverse-engineered). Contains GPS coords, altitude, speed, attitude, battery. DAT files not supported.

**DJI / Litchi CSV** `.CSV` Litchi / DJI exports

CSV with known column patterns: OSD.latitude, datetime(utc), BATTERY.voltage etc. AI maps any unknown column variations automatically.

**Betaflight Blackbox** `.BBL / .BFL` Betaflight / INAV

Text log with header section (H lines) and data section. Attitude in centi-degrees. IMU raw values. GPS if enabled. Ideal for FPV analysis.

**GPX Track** `.GPX` Any GPS logger

Standard GPS Exchange Format XML. Supports waypoints, tracks, routes. Speed from extensions if present. Full coordinate + elevation data.

**KML / KMZ** `.KML / .KMZ` Google Earth

Extracts GPS path from KML coordinates element. KMZ (compressed KML) is also supported.

**Generic CSV (AI)** `.CSV` Any GCS / export tool

AI analyses column headers and content to detect lat, lng, alt, speed, attitude, battery fields — works with any CSV containing location data.

# Section 8a — Skyline .skylog Format Specification

## Overview

The Skyline .skylog format is a proprietary telemetry log produced by the Skyline ground control system (firmware v0.9.x+). It is a plain-text file where each line is a self-contained brace-delimited record. Binary telemetry data is hex-encoded within the text structure. The format was fully reverse-engineered from a real flight log and all field mappings were verified against known values.

> **Verified Against**
> Format specification confirmed by decoding log-20260525-112946.skylog: 98.4-minute flight, ArduPlane V4.3.7 / Skyline V0.3.1, MatekH743 controller, 6S battery (26.13V full, 23.24V minimum), 9,814 clean GPS points, 260 statustext events.

## File Structure

One record per line. Lines prefixed with > are uplink/command records (sent from GCS to vehicle) and are parsed read-only.

```
# Bare Unix timestamp (seconds) — marks time for following records
{1779697786}

# Binary telemetry packet (hex-encoded, 28 bytes)
{tlm:"aa3ec92c5230a0f18f7713d2b0f3961c18896a15fce856ec69e30050"}

# Battery: [raw_int, amps]  ->  voltage = raw / 100.0 V
{vbatt:[2612,0.00]}

# System monitor: [v0,v1,v2,v3,v4,v5,link%,snr,v8,v9,txpow,v11,armed]
{mon:[0,231,0,18,63,41,99,68,42,-1,22,0,2]}

# RC channels: 31-byte SBUS-format frame, channels as uint16 LE
{sbus:"e003de03ac00e003e7035d02..."}

# Home position: lat*1e7, lng*1e7, alt_mm
{home:479616544,359488064,124000}

# Status text: [MAVLink severity, message]
{statustext:[6,"ArduPlane V4.3.7 / Skyline V0.3.1 (208412de)"]}

# Reference waypoint: [id, alt_m, lat, lng, reserved, active_flag]
{ref_set:[85,454.0,47.884617,36.110203,0.0,1]}

# Gimbal: 4 bytes, byte[1]=pan signed int8 deg, byte[2]=tilt signed int8 deg
{gimbal:"02e1e4ff"}

# Current mission waypoint index
{mission_current:5}

# Temperature pair (tenths of degrees C, divide by 10)
{t:[-2355,-2292]}
```

## TLM Packet Binary Layout

The tlm packet is the primary carrier of real-time flight data. All 28-byte packets begin with 0xAA (magic marker). Three subtypes carry GPS coordinates: 0x2C (main, ~65% of packets), 0x28, and 0x29.

| Offset | Size | Type | Field | Notes |
|---|---|---|---|---|
| 0 | 1 byte | uint8 | Magic marker | Always 0xAA |
| 1-3 | 3 bytes | uint24 LE | Rolling counter | Increments each packet |
| 3 | 1 byte | uint8 | Packet subtype | 0x2C=GPS+alt, 0x28/0x29=GPS variants, others=status |
| 4-5 | 2 bytes | int16 LE | Relative altitude | cm above home. ONLY valid when byte[5] < 128. Divide by 100 fo |

| | | | | |
|---|---|---|---|---|
| 6-7 | 2 bytes | int16 LE | Heading/bearing | Encoded bearing data (not always populated) |
| 8-9 | 2 bytes | int16 LE | Ground speed | cm/s. ONLY valid when byte[5] < 128. Divide by 100 for m/s. |
| 10-11 | 2 bytes | int16 LE | Vertical speed | cm/s (signed). Positive = climbing. |
| 12-15 | 4 bytes | int32 LE | Latitude | Degrees * 1e7. Divide by 10,000,000 for decimal degrees. |
| 16-19 | 4 bytes | int32 LE | Longitude | Degrees * 1e7. Divide by 10,000,000 for decimal degrees. |
| 20-25 | 6 bytes | mixed | Additional data | CRC, sequence, or extra telemetry fields |
| 26-27 | 2 bytes | uint16 | CRC/checksum | Packet integrity check |

**Critical: Altitude Validity Filter**

Bytes 4-9 (altitude and speed) are ONLY valid when byte[5] < 128 (high bit clear). When byte[5] >= 128 (values 128-255), bytes 4-11 contain a different internal data structure and must NOT be interpreted as altitude or speed. This was verified by observing that packets with byte[5] in {131,132} produced impossible values (-319m altitude, 306 m/s speed) while byte[5] in {0-4,18,19} produced physically consistent values (0-100m rel altitude, 0-99 m/s speed).

## Battery Voltage Encoding

The vbatt record stores raw integer values where dividing by 100 gives volts. Confirmed: raw value 2612 = 26.12V (6S LiPo fully charged at 4.35V/cell). During flight the value dropped from 2612 to 2331 (23.31V = 3.885V/cell). The second vbatt field is current in amps (stored as float, 0.00 in this sample indicating current sensing was not connected).

## SBUS RC Channel Encoding

SBUS frames are 31 bytes stored as hex. Channels 1-16 are packed as uint16 LE at byte offsets 0, 2, 4, ... 30. The SBUS value range is 172 (minimum) to 1811 (maximum) with 992 as center. To convert to standard PWM microseconds: us = ((raw - 992) / 819.0 * 500) + 1500. Example: 0xe003 = 992 decimal = 1500us (center position).

## Mon[] System Monitor Fields

| Index | Field | Notes |
|---|---|---|
| [6] | Link quality % | 0-100. Percentage of RF link quality. 99 = excellent. |
| [7] | SNR | Signal-to-noise ratio value. |
| [10] | TX power / armed | -9 = vehicle disarmed/on ground. >=0 = armed / in-flight. |
| [12] | Armed flag | 2 = motors armed and active. 0 = disarmed/safe. |

## Parsing Implementation Notes

- Timestamps are Unix epoch seconds. Subtract the first timestamp in the file to get relative flight time in seconds.
- Lines prefixed with > are uplink (GCS-to-vehicle) commands. They use the same record format and can be parsed identically, but their values represent commands sent, not received telemetry.

• The {home:} record should be tracked throughout the file — it may update as the GPS lock improves. Always use the most recent home record to compute ASL altitude.

• GPS coordinates are only valid (non-zero) after the vehicle achieves a GPS fix, typically 1-5 minutes into the log. Filter out points where abs(lat) < 0.5 or abs(lng) < 0.5.

• The {env:} record contains the Skyline firmware version (v field) and device ID (id field). The firmware string from statustext (e.g. 'ArduPlane V4.3.7 / Skyline V0.3.1') gives the full autopilot firmware.

• ref_set records define the mission reference waypoints used for corridor-based navigation. These are not mission_item waypoints but rather area/corridor reference points.

# Section 9 — AI Import Engine Setup

## How the AI Import Engine Works

The import engine uses a two-stage approach: fast heuristic detection first, then Claude AI for ambiguous files. Known formats (including Skyline .skylog) are parsed instantly with no API cost.

| | | |
|---|---|---|
| 1 | **File Upload** | User uploads log file |
| 2 | **Read Sample** | First 8KB read (binary + text) |
| 3 | **Heuristic Check** | Magic bytes, headers, known patterns |
| 4 | **Confidence >= 95%?** | Yes -> skip AI \| No -> call Claude |
| 5 | **Claude API** | Analyse sample, detect format + columns |
| 6 | **Select Parser** | Route to correct parser class |
| 7 | **Parse & Store** | Extract telemetry -> MySQL |
| 8 | **AI Anomaly Scan** | Check for errors/warnings (optional) |

### Getting an Anthropic API Key

```
1. Go to: https://console.anthropic.com/
2. Sign up or log in
3. Navigate to API Keys -> Create Key
4. Copy the key (starts with sk-ant-...)
5. Add to config.local.php:
   define('ANTHROPIC_API_KEY', 'sk-ant-YOUR_KEY');

# Cost estimate: ~0.001 USD per import (very small sample sent)
# AI is OPTIONAL -- all known formats work without it
```

**Without an API Key**

The system works perfectly for ArduPilot BIN, PX4 ULog, MAVLink TLOG, DJI TXT/CSV, Betaflight BBL, GPX, KML, and Skyline .skylog without any AI key. The AI is only called for files where heuristic confidence is below 95% (unknown CSV variants).

# Section 10 — Testing & Verification

## API Health Checks

### Test API is reachable

- GET https://yourdomain.com/api/v1/auth/login → should return 405 (method not allowed)
- POST https://yourdomain.com/api/v1/auth/login with bad creds → should return 401

```
$ curl -X POST https://yourdomain.com/api/v1/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"email":"test@test.com","password":"wrong"}'
# Expected: {"error": "Invalid credentials"}
```

### Test user registration

- POST to /auth/register with email, password, display_name:

```
$ curl -X POST https://yourdomain.com/api/v1/auth/register \
  -H 'Content-Type: application/json' \
  -d '{"email":"pilot@test.com","password":"Test1234!","display_name":"Test Pilot"}'
# Expected: {"token": "...", "user": {...}}
```

### Test file upload

- Upload a sample log file (use any small .csv or .gpx):

```
$ curl -X POST https://yourdomain.com/api/v1/flights \
  -H 'Authorization: Bearer YOUR_TOKEN' \
  -F 'log=@your_logfile.gpx'
# Expected: {"success":true, "flight_id":1, "format":"gpx", "format_confidence":99}
```

# Section 11 — Troubleshooting

### White/blank page after deploying web app

→ All files from web/dist/ were not uploaded to public_html/ root (not a subfolder)
→ The .htaccess file is missing or not being read (mod_rewrite not enabled)
→ Browser cache: press Ctrl+Shift+R (hard refresh)
→ Open browser DevTools → Console for JavaScript errors

### API returns 500 Internal Server Error

→ Check PHP error log in cPanel → Error Logs
→ Verify config.local.php has correct database credentials
→ Ensure MySQL database and user exist and user has full privileges
→ Check UPLOAD_DIR exists and is writable (chmod 755)

### 'Unauthorized' on every API request

→ JWT_SECRET in config.local.php must match between requests
→ Token may have expired (default 72 hours) — log out and back in
→ ALLOWED_ORIGINS must include your domain (for web) and localhost (for dev)

### File upload fails / times out

→ Check PHP limits: upload_max_filesize, post_max_size, max_execution_time
→ In cPanel → Select PHP Version → PHP Options, set all to 512M / 300 seconds
→ Very large BIN files (400MB+) may need max_execution_time = 600

### Mobile app can't connect to server

→ Verify HTTPS is enabled on your domain (HTTP not allowed from mobile apps)
→ Check ALLOWED_ORIGINS includes mobile-side origin or set to ['*'] for testing
→ Ensure API_BASE_URL in mobile/src/api.js is exactly https://yourdomain.com/api/v1
→ No trailing slash in the URL

### Format detected as 'unknown' / parse error

→ Add your Anthropic API key to config.local.php for AI-assisted detection
→ Check server error logs for the specific parse error message
→ Try renaming the file with the correct extension (.bin, .csv, .gpx etc.)
→ Some DJI .DAT files are encrypted and cannot be parsed without DJI SDK

### Database schema import fails in phpMyAdmin

→ Ensure MySQL version is 5.7+ or MariaDB 10.3+
→ The PARTITION BY HASH clause requires InnoDB engine — check engine support
→ Import in parts: first run the CREATE TABLE statements, then the indexes
→ Try removing partition clauses if your host doesn't support partitioning

## Section 12 — Security Hardening

Follow these steps before going live. Default configurations are for easy setup — production systems need additional hardening.

**✓ Use HTTPS everywhere**

Enable SSL via cPanel → SSL/TLS. Let's Encrypt is free. Mobile apps require HTTPS — HTTP will be rejected by iOS and Android.

**✓ Strong JWT secret**

Generate with: openssl rand -hex 64. Must be at least 32 characters. The installer generates this automatically.

**✓ Restrict config.local.php**

The installer sets permissions to 600. Verify with: ls -la public_html/api/config/. The .htaccess also blocks direct web access to .php files in the config/ directory.

**✓ Database user privileges**

Grant only SELECT, INSERT, UPDATE, DELETE on your specific database. Never use the root MySQL user.

**✓ Upload directory location**

The UPLOAD_DIR should be OUTSIDE public_html (e.g. /home/user/uploads/). The installer defaults to ~/uavlogbook_data/uploads/ for this reason.

**✓ Rate limiting**

Consider adding rate limiting to the login endpoint via .htaccess mod_evasive or a cPanel security plugin to prevent brute-force attacks.

**✓ Regular backups**

Set up cPanel automated backups for both the MySQL database and the uploads directory. Flight log files are large — ensure backup storage is adequate.

**✓ PHP version**

Use PHP 8.1+ for best security. In cPanel → Select PHP Version, choose the latest available.

# Section 13 — FPV Video Sync

## Overview

UAVLogBook v1.2 adds FPV video upload and time-synchronisation. Videos are served directly by Apache (HTML5 range-request streaming — no transcoding). The sync engine calculates the offset between video clock and log clock so that every module on the dashboard (map cursor, altitude chart, events timeline) stays locked to the correct video frame.

## How Auto Sync Works

### 1 MP4 creation_time atom

The parser reads the moov/mvhd atom from the MP4/MOV file in pure PHP — no FFmpeg required. This atom stores the recording start time as seconds since 1 January 1904. Subtracting 2,082,844,800 converts to Unix time. The offset = video_start_unix_ms – log_start_unix_ms. Confidence: 85%. Works with GoPro, DJI, Insta360, RunCam, Sony.

### 2 FFprobe metadata (if installed)

If FFprobe is available on the server, it is used as a more reliable fallback. FFprobe reads creation_time from format tags and per-stream tags, also extracting duration, resolution, FPS, and codec. To install: sudo apt install ffmpeg (Ubuntu/Debian) or use cPanel Softaculous.

### 3 Filename timestamp parsing

If no metadata timestamp is found, the filename is scanned with regex patterns for common camera naming conventions: DJI (DJI_20250525_143022), GoPro (GH010123_20250525_143022), RunCam (RC_2025-05-25_14-30-00), and generic ISO-style (2025-05-25T14:30:22). Confidence: 65%. Manual correction recommended.

### 4 Manual correction slider

The UI provides a ±30 second range slider with selectable step sizes (33ms = 1 frame at 30fps, 100ms, 500ms, 1000ms). The user plays the video to a recognisable event (takeoff, sharp turn, landing), reads the log-time overlay on the video player, and nudges the correction until aligned. The correction is saved per-video in the database and persists across sessions.

## Sync Offset Formula

```
# The single stored value:
effective_offset_ms = sync_offset_ms (auto) + manual_correction_ms (user)

# How video position maps to log position:
log_time_ms = video_time_ms + effective_offset_ms
video_time_ms = log_time_ms - effective_offset_ms

# Examples:
# Video started 5s AFTER log started  -> offset = +5000ms
#   At video 0:00 -> log position is 0:05
#   At video 1:30 -> log position is 1:35

# Video started 3s BEFORE log started -> offset = -3000ms
#   At video 0:03 -> log position is 0:00
#   At video 1:30 -> log position is 1:27

# Positive correction: video is running ahead of log (add ms to offset)
# Negative correction: video is running behind log (subtract ms from offset)
```

# Server Setup for Video

## Step 1 — Create video upload directory

- Create: /home/youraccount/uavlogbook_data/uploads/videos/
- Set permissions to 755 (writable by web server)
- The installer script does this automatically

## Step 2 — Configure cPanel PHP upload limits

- Videos can be up to 8 GB — PHP defaults are far too small
- In cPanel → Select PHP Version → PHP Options:
- upload_max_filesize = 8192M
- post_max_size = 8192M
- max_execution_time = 600
- max_input_time = 600
- Alternatively, use chunked upload (future enhancement)

## Step 3 — Enable Apache range requests

- The .htaccess already includes: Header always set Accept-Ranges bytes
- This is required for HTML5 video seeking to work correctly
- Without it, the browser cannot seek within the video

## Step 4 — Optional: Install FFmpeg for thumbnails and metadata

- sudo apt install ffmpeg (Ubuntu/Debian via SSH)

- Or use cPanel Softaculous / EasyApache FFmpeg module
- Without FFmpeg: metadata extraction still works via pure-PHP MP4 atom parser
- With FFmpeg: also gets thumbnails, and more reliable metadata (all formats)

## Supported Video Formats

| Format | Extension(s) | Notes |
|---|---|---|
| MPEG-4 | .mp4 / .m4v | Best support. H.264/H.265. Used by DJI, GoPro, RunCam, phones. |
| QuickTime | .mov | Apple/GoPro native. Same container as MP4, full metadata support. |
| AVI | .avi | Older format. Limited metadata. Filename sync recommended. |
| Matroska | .mkv | Open container. Metadata support varies by encoder. |
| WebM | .webm | Browser-native format. Good for streaming. |
| MPEG | .mpg / .mpeg | Legacy. Minimal metadata. |
| 3GPP | .3gp | Mobile/action cam format. |

**Video Storage**

Videos are stored outside public_html in the uavlogbook_data/uploads/videos/ directory. A symlink or Apache Alias directive serves them as /uploads/videos/ in the web root. The installer creates this automatically. Videos are never re-encoded — they are served as-is using HTTP range requests for efficient seeking.

# Appendix A — File Structure Reference

## Package Contents

| | |
|---|---|
| uavlogbook/ | Root directory |
| ■■■ README.md | Overview and quick start |
| ■■■ server/ | PHP backend — deploy to public_html/api/ |
| ■ ■■■ api/index.php | Main REST API router (all endpoints) |
| ■ ■■■ api/processor.php | Flight processing orchestrator |
| ■ ■■■ ai/import_engine.php | AI format detection + anomaly analysis |
| ■ ■■■ config/config.php | Configuration loader |
| ■ ■■■ config/db.php | PDO database connection + batch helpers |
| ■ ■■■ db/schema.sql | Full MySQL schema — import via phpMyAdmin |
| ■ ■■■ middleware/auth.php | JWT auth, password hashing, UUID gen |
| ■ ■■■ parsers/ardupilot_bin.php | ArduPilot DataFlash .BIN parser |
| ■ ■■■ parsers/multi_parsers.php | PX4, DJI, GPX, Betaflight parsers |
| ■ ■■■ parsers/skyline_parser.php | Skyline .skylog parser (reverse-engineered) |
| ■ ■■■ api/video_handler.php | Video upload, MP4 atom parser, auto time-sync engine |
| ■ ■■■ .htaccess | URL routing + security rules |
| ■■■ web/ | React web app — build then upload dist/ |
| ■ ■■■ src/App.jsx | Main application with all modules/views |
| ■ ■■■ src/VideoSync.jsx | FPV video player, upload, and time-sync UI module |
| ■ ■■■ src/store.js | Zustand state management + module registry |
| ■ ■■■ src/api.js | Axios API client with JWT interceptors |
| ■ ■■■ package.json | Dependencies |
| ■ ■■■ vite.config.js | Build configuration |
| ■ ■■■ .env.example | Copy to .env.local and set VITE_API_URL |
| ■■■ mobile/ | React Native app — build with Expo |
| ■ ■■■ App.js | Main app with navigation and all screens |
| ■ ■■■ src/api.js | Mobile API client + SecureStore auth |
| ■ ■■■ app.json | Expo configuration |
| ■ ■■■ package.json | Mobile dependencies |
| ■■■ installer/ | Automated installation helpers |
| ■■■ install_server.sh | Bash installer for Linux/cPanel SSH |
| ■■■ build_web.sh | macOS/Linux web build script |
| ■■■ build_web.ps1 | Windows PowerShell build script |